

Lecture 5 — Two-Dimensional Viewing and Clipping I: Window, Viewport, Point Clipping, and Line Clipping

Lecture focus: How a selected region of a 2D world is mapped to a display area, and how invisible objects or object parts are removed using clipping. This lecture focuses on window-to-viewport mapping, point clipping, and line clipping with Cohen–Sutherland and Liang–Barsky ideas.

Learning outcomes

1. Explain the difference between a **window** and a **viewport**.
2. Explain why viewing transformation is needed in 2D graphics.
3. Derive and use the window-to-viewport mapping equations.
4. Explain the purpose of clipping.
5. Perform simple point clipping.
6. Explain region codes in the Cohen–Sutherland line clipping algorithm.
7. Apply trivial accept and trivial reject tests.
8. Explain the iterative clipping step of Cohen–Sutherland.
9. Describe the basic idea of Liang–Barsky line clipping.
10. Relate viewing and clipping to maps, games, CAD, GUI systems, and modern rendering pipelines.

1. Why viewing and clipping come after transformations

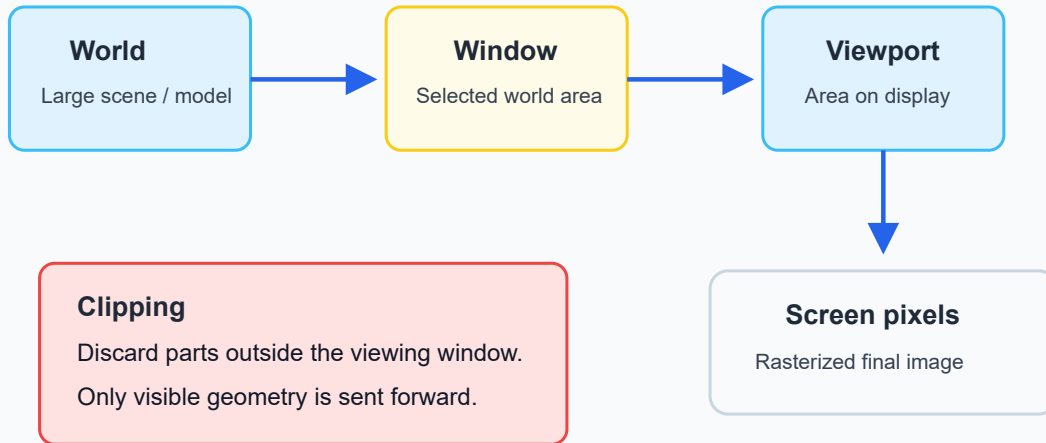
In Lecture 4, we discussed how to move, rotate, scale, shear, and reflect objects. That tells us how objects are placed in a scene.

But after building a scene, another question appears:

Which part of the scene should be visible on the screen?

A large 2D world may contain many objects. The display may show only a portion of it. For example, a map app shows only the current map area, a game camera shows only part of a level, and a CAD program lets the user zoom into part of a drawing.

2D Viewing and Clipping: From World to Screen



The central pipeline is:

```
World coordinates → Window selection → Clipping → Viewport mapping → Screen pixels
```

2. Window and viewport

Two terms are extremely important.

Window

The **window** is the selected rectangular region in **world coordinates**.

It answers:

Which part of the world do we want to see?

Example:

```
world window: x = 100 to 300, y = 50 to 200
```

Viewport

The **viewport** is the rectangular region on the **display device** where the window will be shown.

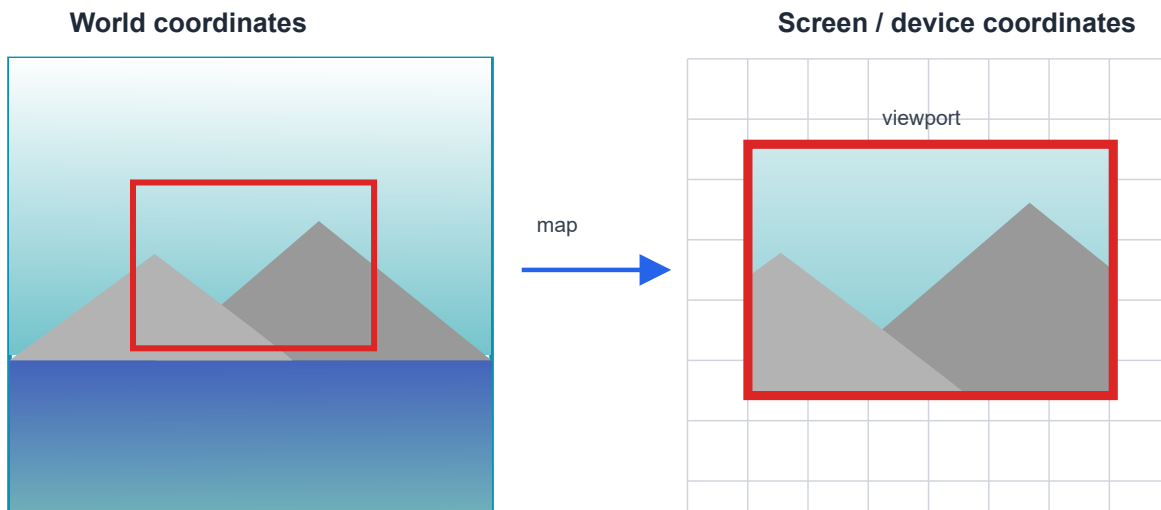
It answers:

Where on the screen should we display the selected world region?

Example:

```
screen viewport: x = 20 to 620, y = 40 to 440
```

Window-to-viewport mapping



The selected world window is scaled and translated to fit the viewport.

Simple analogy

Think of the world as a large painting. The **window** is the part of the painting you photograph. The **viewport** is the area of the screen where you show that photograph.

3. Window-to-viewport mapping

Window-to-viewport mapping converts a point from world coordinates to viewport coordinates.

Suppose the world window is:

```
xwmin, xwmax  
ywmin, ywmax
```

and the viewport is:

```
xvmin, xmax  
yvmin, ymax
```

The mapping uses scaling and translation.

Window-to-viewport equations

Window

```
xwmin, xmax  
ywmin, ymax
```

Viewport

```
xvmin, xmax  
yvmin, ymax
```



Mapping formula

```
sx = (xmax-xvmin) / (xwmax-xwmin)  
sy = (ymax-yvmin) / (ywmax-ywmin)  
  
xv = xvmin + (xw-xwmin) sx  
yv = yvmin + (yw-ywmin) sy
```

If screen y increases downward, the y formula may need inversion.

The scale factors are:

```
sx = (xmax - xvmin) / (xwmax - xwmin)  
sy = (ymax - yvmin) / (ywmax - ywmin)
```

Then a world point (x_w, y_w) maps to viewport point (x_v, y_v) :

```
xv = xvmin + (xw - xwmin) sx  
yv = yvmin + (yw - ywmin) sy
```

Important screen-coordinate warning

In the usual mathematical convention, y increases upward. In many screen systems, y increases downward from the top-left corner. So some systems use a y-mapping like:

```
yv = ymax - (yw - ywmin) sy
```

Always check the coordinate convention.

4. Worked example: window-to-viewport mapping

Suppose the window is:

```
xwmin = 0,    xwmax = 100  
ywmin = 0,    ywmax = 100
```

and the viewport is:

```
xvmin = 200, xvmax = 600  
yvmin = 100, yvmax = 500
```

Find the viewport coordinates of world point:

```
P = (25, 75)
```

First compute scale:

```
sx = (600 - 200) / (100 - 0) = 4  
sy = (500 - 100) / (100 - 0) = 4
```

Now map the point:

```
xv = 200 + (25 - 0) × 4 = 300  
yv = 100 + (75 - 0) × 4 = 400
```

So:

```
(25, 75) → (300, 400)
```

This example assumes both coordinate systems increase y in the same direction.

5. Aspect ratio problem

The **aspect ratio** is the ratio of width to height.

If the window and viewport have different aspect ratios, the image may be stretched.

Example:

```
window: 100 × 100 → aspect ratio 1:1  
viewport: 400 × 200 → aspect ratio 2:1
```

A circle in the window may appear as an ellipse in the viewport.

How to avoid distortion

We can preserve aspect ratio by using the same scale factor in both directions:

```
s = min(sx, sy)
```

Then we may leave some unused space in the viewport, similar to letterboxing in video.

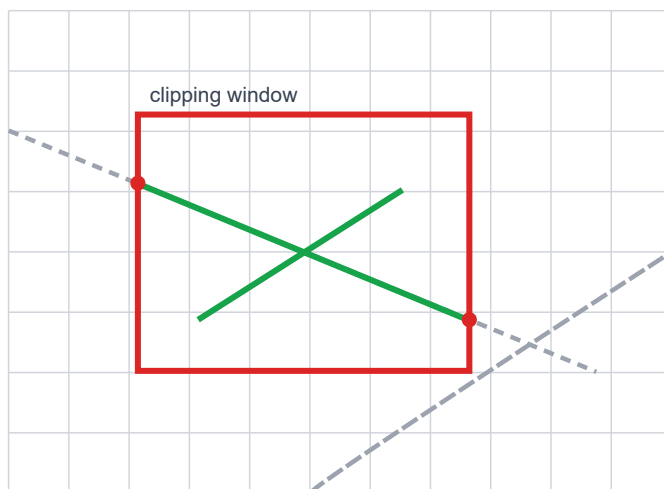
Trivia

When you watch a movie on a screen with a different aspect ratio, black bars may appear. This is the same visual idea: preserve the original shape rather than stretching the image.

6. What is clipping?

Clipping removes parts of objects that lie outside the viewing window.

Clipping: keep only what can be seen



Outside part: discarded
Inside part: drawn
Intersections: computed
Result: less drawing work

Why do we clip?

1. To avoid drawing invisible geometry.

2. To reduce unnecessary computation.
3. To make sure objects do not draw outside their allowed region.
4. To prepare geometry for later rasterization.

Without clipping

A line or polygon outside the visible area may still be processed unnecessarily. A user-interface component may draw outside its panel.

With clipping

Only the visible parts are passed forward.

7. Types of clipping

Common clipping problems include point clipping, line clipping, polygon clipping, curve clipping, text clipping, and viewport/scissor clipping.

This lecture focuses mainly on **point clipping** and **line clipping**. Polygon clipping will be discussed in the next lecture.

8. Point clipping

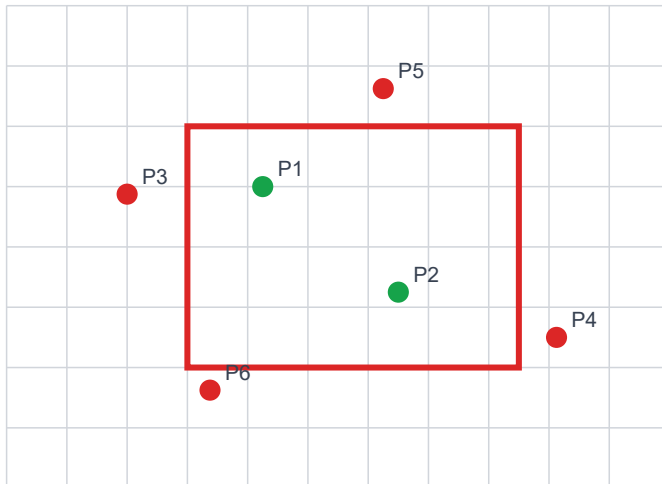
Point clipping is the simplest case.

A point (x, y) is visible if it lies inside the clipping window:

$$\begin{aligned}x_{wmin} &\leq x \leq x_{wmax} \\ y_{wmin} &\leq y \leq y_{wmax}\end{aligned}$$

Otherwise, the point is invisible and is rejected.

Point clipping



A point is visible if:

$$x_{wmin} \leq x \leq x_{wmax}$$

$$y_{wmin} \leq y \leq y_{wmax}$$

Otherwise it is clipped.

Pseudocode

```
PointClip(x, y):  
    if  $x_{wmin} \leq x \leq x_{wmax}$  and  $y_{wmin} \leq y \leq y_{wmax}$ :  
        accept point  
    else:  
        reject point
```

Example

Window:

```
x = 10 to 50  
y = 20 to 80
```

Point:

```
P = (30, 70)
```

Since:

```
10 ≤ 30 ≤ 50  
20 ≤ 70 ≤ 80
```


the point is accepted.

Point:

```
Q = (5, 30)
```

Since $5 < 10$, the point is rejected.

9. Line clipping: why it is harder

A line segment has two endpoints.

A line can be completely inside the window, completely outside the window, or partly inside and partly outside.

The third case is the interesting one. We must compute the intersection between the line and the clipping window.

A good line clipping algorithm should quickly determine:

```
accept → reject → or clip and continue
```

10. Cohen–Sutherland line clipping algorithm

The Cohen–Sutherland algorithm is a classic line clipping algorithm for rectangular windows.

Its main idea is to assign each endpoint a 4-bit region code describing where the point lies relative to the clipping window.

Cohen–Sutherland region codes



Each endpoint receives a 4-bit code relative to the clipping window.

The common bit meanings are:

```
TOP      = 1000
BOTTOM   = 0100
RIGHT    = 0010
LEFT     = 0001
```

The inside region has code:

```
0000
```

How to compute a region code

For a point (x, y) :

```
code = 0000

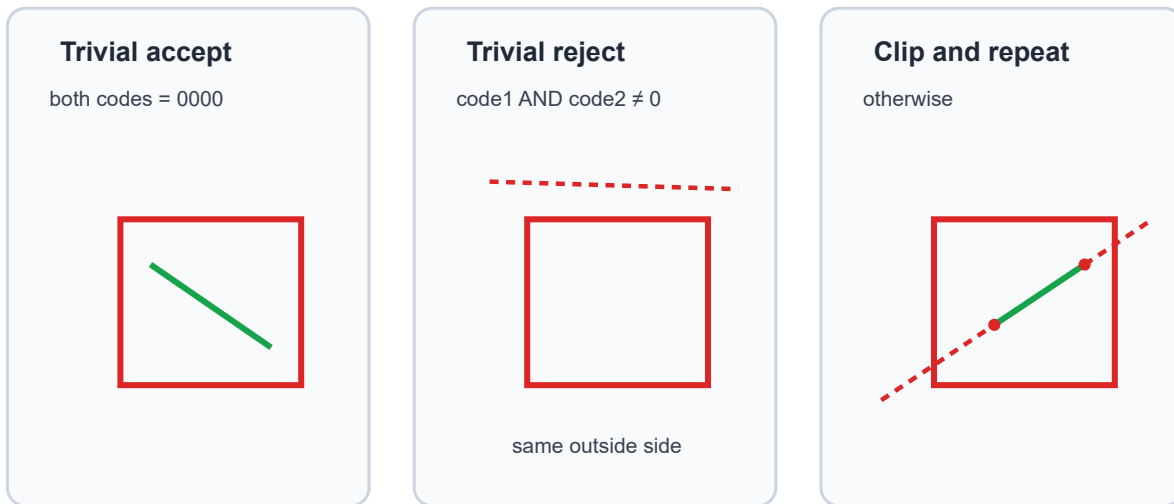
if y > ymax: set TOP bit
if y < ymin: set BOTTOM bit
if x > xmax: set RIGHT bit
if x < xmin: set LEFT bit
```

The exact top/bottom comparison may depend on the coordinate convention, but the idea is the same.

11. Cohen–Sutherland: three cases

After computing endpoint codes, there are three main cases.

Cohen–Sutherland line clipping: three main cases



Case 1: Trivial accept

If both endpoint codes are 0000, then both endpoints are inside.

```
code1 OR code2 == 0
```

The whole line is accepted.

Case 2: Trivial reject

If the bitwise AND of the two codes is not zero:

```
code1 AND code2 != 0
```

then both endpoints are outside on the same side of the window. The line is rejected.

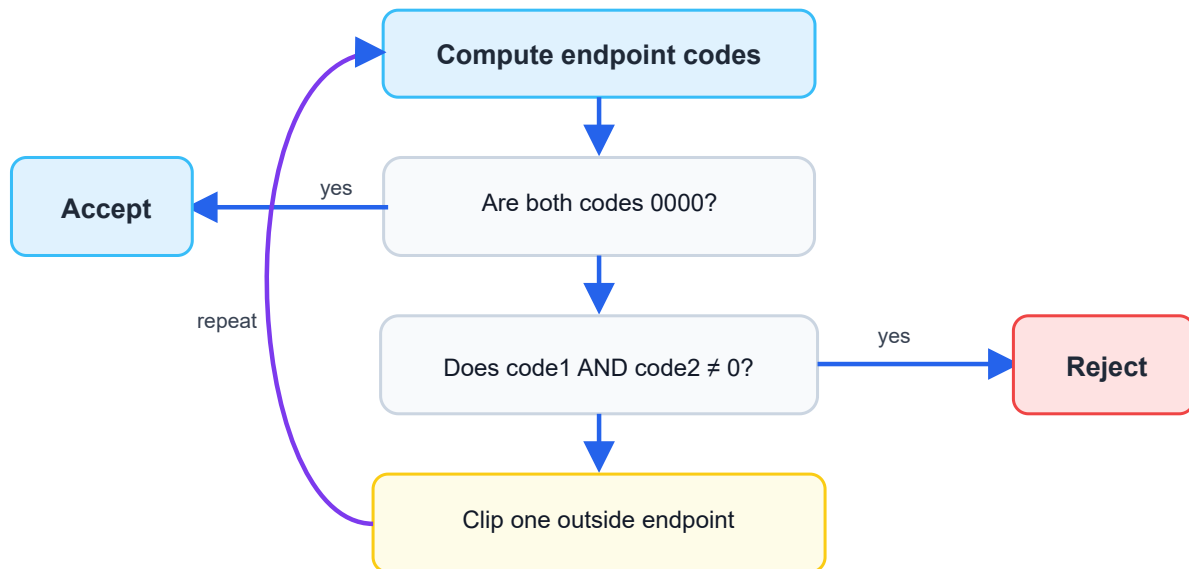
Case 3: Clip and repeat

If neither of the above cases applies, the line may cross the window.

We choose one outside endpoint, find its intersection with the relevant window boundary, replace that endpoint with the intersection point, recompute the code, and repeat.

12. Cohen–Sutherland algorithm flow

Cohen–Sutherland algorithm flow



Pseudocode

```
CohenSutherlandClip(P1, P2):
    code1 = ComputeCode(P1)
    code2 = ComputeCode(P2)

    while true:
        if code1 == 0 and code2 == 0:
            accept line
            break

        else if (code1 AND code2) != 0:
            reject line
            break

        else:
            choose an endpoint that is outside
            find intersection with an appropriate boundary
            replace that endpoint with the intersection point
            recompute its code
```

Boundary intersection formulas

For a line from (x_1, y_1) to (x_2, y_2) , the slope is:

$$m = (y_2 - y_1) / (x_2 - x_1)$$

Intersection with top boundary $y = y_{\max}$:

$$\begin{aligned} x &= x_1 + (y_{\max} - y_1) / m \\ y &= y_{\max} \end{aligned}$$

Intersection with bottom boundary $y = y_{\min}$:

$$\begin{aligned} x &= x_1 + (y_{\min} - y_1) / m \\ y &= y_{\min} \end{aligned}$$

Intersection with right boundary $x = x_{\max}$:

$$\begin{aligned} y &= y_1 + m (x_{\max} - x_1) \\ x &= x_{\max} \end{aligned}$$

Intersection with left boundary $x = x_{\min}$:

$$\begin{aligned} y &= y_1 + m (x_{\min} - x_1) \\ x &= x_{\min} \end{aligned}$$

Practical note

For vertical lines, slope m may be undefined. Robust implementations handle vertical and horizontal lines carefully.

13. Worked example: Cohen–Sutherland logic

Suppose the clipping window is:

$$\begin{aligned} x_{\min} &= 10, \quad x_{\max} = 50 \\ y_{\min} &= 10, \quad y_{\max} = 40 \end{aligned}$$

Line endpoints:

```
P1 = (5, 20)
P2 = (30, 30)
```

For `P1 = (5, 20)`:

```
x < xmin
```

so it is to the left of the window:

```
code1 = LEFT = 0001
```

For `P2 = (30, 30)`, the point is inside:

```
code2 = 0000
```

Now:

```
code1 OR code2 != 0
```

so we cannot trivially accept.

Also:

```
code1 AND code2 = 0000
```

so we cannot trivially reject.

Therefore, we clip the outside endpoint `P1` against the left boundary:

```
x = xmin = 10
```

Then compute the corresponding y-value and replace `P1`.

After this, both endpoints may become inside, so the clipped line is accepted.

14. Why bitwise operations are useful here

Cohen–Sutherland is elegant because it turns geometry into quick bit tests.

- `OR` test checks whether both endpoints are inside.

- **AND** test checks whether endpoints are on the same outside side.
- Only if needed, we compute intersection.

Trivia

Bitwise logic was especially attractive on early computers because it was very fast. This is one of the reasons Cohen–Sutherland became a famous textbook algorithm.

15. Liang–Barsky line clipping: short introduction

Another important line clipping algorithm is the **Liang–Barsky algorithm**.

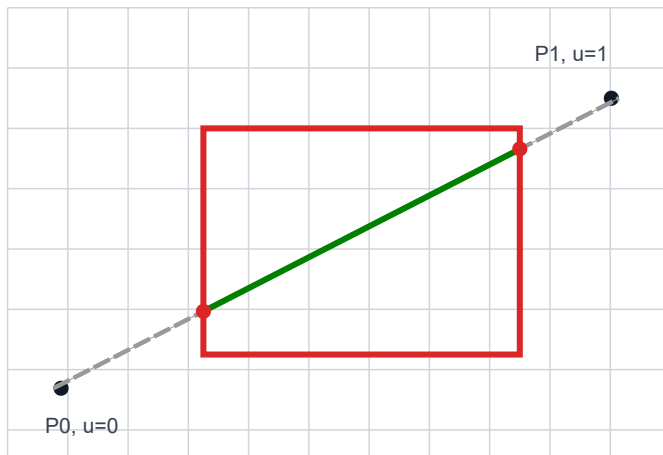
Instead of using region codes, it represents the line segment parametrically:

$$P(u) = P_0 + u(P_1 - P_0)$$

where:

$$0 \leq u \leq 1$$

Liang–Barsky: clipping by line parameter



Line equation:
 $P(u) = P_0 + u(P_1 - P_0)$
 $0 \leq u \leq 1$
 Find entering u
 and exiting u .

The algorithm finds the interval of **u** values for which the line is inside the clipping window.

For example:

```
u_enter = 0.25
u_exit  = 0.80
```

Then the visible part is:

```
P(0.25) to P(0.80)
```

Why Liang–Barsky is useful

Liang–Barsky can be more efficient because it calculates entering and exiting parameter values directly, often avoiding repeated clipping steps.

What to remember

For this course level, remember the conceptual difference:

- Cohen–Sutherland uses **region codes**.
- Liang–Barsky uses the **parameter interval of a line**.

16. Cohen–Sutherland vs Liang–Barsky

Cohen–Sutherland vs Liang–Barsky

Aspect	Cohen–Sutherland	Liang–Barsky
Main idea	Region codes	Line parameter u
Strength	Very visual and easy to teach	Often fewer intersection calculations
Best for	Understanding accept/reject logic	Efficient rectangular clipping
Computation	May clip repeatedly	Computes visible parameter interval
Classroom value	Excellent first line clipping algorithm	Good follow-up optimization

Both solve the same problem: find the visible part of a line segment inside a rectangular window.

Cohen–Sutherland is often easier to understand first because the 4-bit region codes are visual.

Liang–Barsky is mathematically cleaner once we are comfortable with parametric lines.

Both algorithms solve the same problem:

Find the visible part of a line segment inside a rectangular clipping window.

17. Clipping in user interfaces

Clipping is not only for mathematical line segments.

In graphical user interfaces, every panel, button, card, and text box may have a clipping boundary.

Example:

- A scrollable list clips items outside the visible scroll area.
- A text editor clips long lines.
- A browser clips content inside cards or containers.
- A game clips UI components inside the screen.

Practical example

When you scroll a webpage, the browser does not draw the entire infinite document into the visible screen. It computes what part should be visible and paints that region.

18. Applications of viewing and clipping

Where viewing and clipping appear

Maps

Pan and zoom by changing the viewing window.

Games

Camera view decides what part of the world is visible.

CAD

Zoom into drawings and clip outside geometry.

GUI systems

Draw only inside panels, buttons, and scroll areas.

Text editors

Clip text and cursor inside the visible editor region.

GPUs

Clipping is part of the rendering pipeline before rasterization.

Trivia: When you scroll a webpage, the browser is deciding what visible parts should be painted.

Maps

Changing the map window creates pan and zoom effects.

Games

The camera view selects which part of the game world is visible.

CAD

Engineers zoom into parts of a large drawing. The viewport displays the selected region.

GUI systems

A button or panel should not draw outside its own area.

Text editors

Text and cursors are clipped inside the editor area.

GPUs

Clipping is part of the rendering pipeline before rasterization.

19. Common mistakes

1. Confusing window and viewport.
 2. Forgetting that the window is in world coordinates and the viewport is in device coordinates.
 3. Ignoring y-axis direction differences between mathematics and screens.
 4. Stretching the image accidentally by using unequal scale factors.
 5. Thinking clipping and mapping are the same thing.
 6. Forgetting the **AND** test in Cohen–Sutherland.
 7. Rejecting a line just because one endpoint is outside.
 8. Computing line intersections before checking trivial accept/reject cases.
 9. Using slope formulas carelessly for vertical lines.
 10. Mixing up TOP and BOTTOM bits under different coordinate conventions.
-

20. Short exam-style questions

1. Define window and viewport.
2. Why is window-to-viewport mapping needed?

3. Derive the x-coordinate mapping equation from world window to viewport.
 4. What is clipping?
 5. Write the condition for accepting a point inside a rectangular clipping window.
 6. What is a region code in the Cohen–Sutherland algorithm?
 7. What does code `0000` mean?
 8. Explain the trivial accept test.
 9. Explain the trivial reject test.
 10. How does Liang–Barsky differ from Cohen–Sutherland?
-

21. Practice problems

Problem 1

A world window is:

```
xwmin = 0, xwmax = 200  
ywmin = 0, ywmax = 100
```

A viewport is:

```
xvmin = 100, xvmax = 500  
yvmin = 50, yvmax = 250
```

Map world point `(50, 25)` to viewport coordinates.

Problem 2

For window:

```
xmin = 10, xmax = 100  
ymin = 20, ymax = 80
```

Decide whether each point is accepted or rejected:

```
A(30, 40)  
B(5, 50)  
C(90, 100)  
D(100, 80)
```

Problem 3

For the same window, compute Cohen–Sutherland region codes for:

```
P1(5, 30)
P2(50, 90)
P3(120, 10)
P4(40, 40)
```

Problem 4

A line has endpoint codes:

```
code1 = 1000
code2 = 1010
```

Can it be trivially rejected? Explain.

Problem 5

A line has endpoint codes:

```
code1 = 0001
code2 = 0010
```

Can it be trivially accepted or rejected immediately? Explain what the algorithm should do next.

22. Lecture summary

Lecture 5 summary

● Window	The selected region in world coordinates.
● Viewport	The region on the display where the window is mapped.
● Mapping	Uses scaling and translation from world to device coordinates.
● Clipping	Removes invisible geometry outside the window.
● Point clipping	Simple range test.
● Cohen–Sutherland	Uses 4-bit region codes for line clipping.
● Liang–Barsky	Uses line parameters to find visible interval.

Big idea: viewing selects what we want to see; clipping prevents drawing what cannot be seen.

The big takeaways are:

- A **window** is the selected area in world coordinates.
- A **viewport** is the display area where the selected world region is shown.
- Window-to-viewport mapping is mainly scaling plus translation.
- Clipping removes invisible geometry.
- Point clipping is a simple inside/outside test.
- Cohen–Sutherland line clipping uses 4-bit region codes.
- Trivial accept and reject tests make Cohen–Sutherland efficient.
- Liang–Barsky line clipping uses a parametric line and visible **u** interval.
- Viewing and clipping are essential in maps, games, CAD, GUIs, and rendering pipelines.

23. Final trivia box

- The idea of a “camera” in games is closely related to the 2D viewing window.
- A minimap in a game is basically another viewport showing a different view of the world.
- Scrollable UI panels use clipping constantly.
- In modern graphics APIs, a fast rectangular clipping operation is often called a **scissor test**.
- Clipping is one of the reasons graphics systems do not waste time drawing everything everywhere.